

BuildTrust

Documento di Architettura Tecnica

Versione 3.0 — Marzo 2026

Classificazione: **Confidenziale**

Redazione: Arxcode

Data: 28 Marzo 2026

Indice

- Sommario Esecutivo
- Contesto di Dominio e Problema
 - Il dominio: monitoraggio cantieri infrastrutturali
 - La complessità tecnica del problema
- Principi Architetturali Fondamentali
- Scelte Architetturali: Perché e Come
 - CQRS — Command Query Responsibility Segregation
 - Il problema
 - La soluzione
 - Perché non un approccio più semplice
 - Event-Driven Architecture e CDC con Debezium
 - Il problema dell'accoppiamento
 - Change Data Capture: perché il WAL e non eventi applicativi
 - RedPanda come Event Broker
 - Architettura Esagonale (Ports & Adapters)
 - Il problema del lock-in tecnologico
 - La soluzione: Ports & Adapters
 - PostgreSQL: Single Database, Multi-Schema, Row Level Security
 - Perché non un database per microservizio
 - Row Level Security per il multi-tenancy
 - Hasura GraphQL Engine
 - Perché non un API layer custom
 - Keycloak per Identity & Access Management
 - Perché non Auth0, Firebase Auth o Cognito
 - Come funziona l'isolamento multi-tenant
 - LLM Service come Gateway AI Unificato
 - Il problema dell'AI distribuita
 - La soluzione: stateless gateway
- Architettura di Sistema: Vista d'Insieme
 - I Sette Layer
 - I Tre Canali di Ingestion
 - Canale 1: File Upload → Bucket Watcher → Parser
 - Canale 2: OPC UA → Production Ingestion
 - Canale 3: Input Manuale (Hasura Mutations)
- Struttura dei Microservizi: Python e FastAPI
 - Template Esagonale Comune
 - MS-01 · BIM Parser
 - MS-02 · Contract Parser

- MS-03 · BOQ Parser
- MS-04 · Gantt Parser
- MS-05 · Production Ingestion
- MS-06 · SAL Engine
- MS-07 · LLM Service
- MS-08 · ACDAT Connector
- MS-09 · Read Model Projector
- MS-10 · Notification Service
- MS-11 · External Sync Service
- Modello Dati: Otto Schemi Core di Dominio
 - Filosofia del modello dati
 - Schema BIM — La Struttura Portante
 - Schema Process — Contratto e Milestone
 - Schema BOQ — Copertura Economica
 - Schema Production — I Dati dall'Impianto
 - Schema Progress e SAL — Il Cuore del Business
 - Schema Quality — Verifica e Blocco
- SAL Engine: Il Cuore Computazionale
 - Architettura del SAL Engine
 - I Sei Passaggi di Validazione
- Integrazioni Esterne
 - ACDAT: il CDE Ufficiale Italiano
 - MS-11 External Sync Service: il Gateway verso l'Ecosistema
 - I Quattro Cluster di Integrazione
 - Cluster A — BIM Authoring Tools
 - Cluster B — Cloud CDE
 - Cluster C — Construction PM
 - Cluster D — PLM / Enterprise
 - Schema PostgreSQL external
 - I Tre Pattern di Integrazione
 - Adapter selezione via ENV
- Sicurezza e Multi-Tenancy
 - I Tre Livelli di Isolamento
 - Gestione dei Ruoli
 - Sicurezza del Canale OPC UA
- Flusso UI: Le 9 Fasi
 - Riepilogo delle 9 Fasi
 - Principi di UX
- Dimensionamento e Deploy
 - GCP come Target di Riferimento
- Modello Dati: Esempio Completo
- Evoluzioni Future

- Glossario
- Nice to Have — MCP Server: Assistente AI Conversazionale
 - Il Problema che Risolve
 - Architettura del Sistema MCP
 - Componenti
 - Flusso di una Domanda
 - Perché MCP e non function calling diretto
 - Perché Chat BFF separato dall'MCP Server
 - Catalogo Tool MCP (18 tool su 6 domini)
 - Dominio Avanzamento / SAL
 - Dominio BIM / WBS
 - Dominio Produzione
 - Dominio Qualità
 - Dominio Contratto / Processo
 - Dominio Portfolio / Cross-progetto
 - Il Tool Composito: get_sal_release_readiness
 - Modello di Sicurezza
 - Propagazione JWT e Tenant
 - Filtraggio Tool per Ruolo
 - Protezione Dati verso LLM Esterno
 - Widget React: Suggerimenti Contestuali per Schermata

Sommario Esecutivo

BuildTrust è una piattaforma per il monitoraggio dell'avanzamento lavori in progetti infrastrutturali complessi. Il sistema integra dati BIM, contratti d'appalto, computi metrici, dati di produzione industriale e conferme di cantiere per calcolare in modo automatico e verificabile lo **Stato di Avanzamento Lavori (SAL)** e abilitare i pagamenti contrattuali.

Il principio fondamentale che guida ogni scelta architettuale è uno solo: **la WBS contenuta nel modello BIM è la Single Source of Truth**. Non esistono dati validi nel sistema che non siano ancorati ad almeno un elemento della WBS. Questa regola non è un vincolo tecnico ma una scelta di dominio: in un progetto infrastrutturale, ogni euro pagato deve essere giustificato da un'attività misurabile e verificabile, non da una dichiarazione.

Questo documento descrive in dettaglio l'architettura tecnica del sistema, spiegando non solo cosa è stato scelto ma *perché*, quali alternative sono state scartate e quali problemi ogni scelta risolve. È destinato a chi deve comprendere, estendere o mantenere la piattaforma.

Nota d'uso. Le strutture di directory, gli snippet di codice, gli endpoint API e i modelli dati riportati in questo documento sono **indicazioni di progettazione**, non implementazioni definitive. Rappresentano il punto di partenza per la fase di sviluppo e dovranno essere validati sul campo, adattati alle specificità di ogni ambiente e sono soggetti a variazioni nel corso dell'implementazione. Nessun elemento tecnico descritto è da considerarsi vincolante fino alla validazione in un contesto reale.

Contesto di Dominio e Problema

Il dominio: monitoraggio cantieri infrastrutturali

Un progetto infrastrutturale complesso — un viadotto, un tunnel, un edificio industriale — genera ogni giorno una quantità enorme di dati eterogenei: il modello BIM con le geometrie e le quantità degli elementi costruttivi, il contratto d'appalto con le condizioni di pagamento e le penali, il computo metrico con la valorizzazione economica, i dati di produzione dagli impianti (es. l'impianto del calcestruzzo che invia telemetria via OPC UA), le bolle di consegna (DDT), i certificati di qualità dei materiali, le conferme di completamento fase da parte del capocantiere.

Senza un sistema integrato, il calcolo di un SAL richiede che il Direttore Lavori o il RUP incroci manualmente tutti questi dati su fogli Excel, PDF e sistemi separati. Il processo richiede giorni di lavoro, è soggetto a errori di trascrizione e non produce una traccia verificabile. In caso di contestazione contrattuale, non è possibile ricostruire con certezza la catena che porta da un batch di calcestruzzo prodotto all'importo certificato nel SAL.

La complessità tecnica del problema

Il problema non è solo raccogliere dati: è garantire che siano **coerenti, verificabili e ancorati alla struttura contrattuale**. Questo crea tre sfide tecniche distinte che guidano l'intera architettura:

Sfida 1 — Eterogeneità delle sorgenti. I dati arrivano da canali completamente diversi per natura e temporalità: file caricati manualmente (BIM, contratto, computo), stream real-time da impianti industriali via OPC UA, input manuali dell'operatore via web app. Un'architettura che tratta queste sorgenti allo stesso modo non funziona; servono canali di ingestion specializzati.

Sfida 2 — Pattern di lettura radicalmente diversi da quelli di scrittura. I dati vengono scritti per dominio (un parser BIM scrive solo entità BIM, un parser contratto scrive solo clausole), ma vengono letti in modo trasversale: il SAL Engine deve aggregare in una singola operazione quantità BIM, dati di produzione, voci di computo, stati di avanzamento e certificati di qualità. Se si usa lo stesso modello di dati per leggere e scrivere, le query di lettura diventano join complessi su 15+ tabelle che richiedono secondi. Questo giustifica il pattern CQRS.

Sfida 3 — Accoppiamento e reattività. Quando un nuovo batch di produzione viene registrato, devono reagire il SAL Engine (ricalcolo dell'avanzamento), il Read Model Projector (aggiornamento dashboard), il Reconciler WBS-BOQ (verifica copertura economica) e potenzialmente il sistema ACDAT. Se questi componenti si chiamano direttamente uno l'altro, il sistema diventa un monolite distribuito fragile. Serve un'architettura a eventi.

Principi Architeturali Fondamentali

I cinque principi seguenti non sono linee guida generiche: sono decisioni vincolanti che hanno influenzato ogni scelta tecnica del sistema.

Single Source of Truth sulla WBS. La Work Breakdown Structure, contenuta nel modello BIM/Revit, è l'unico riferimento per tutte le attività del progetto. Ogni dato — un batch di produzione, una voce di computo, un certificato di qualità, un DDT — deve essere associato a uno o più elementi WBS. Non esistono dati orfani. Questa regola garantisce che il SAL sia sempre calcolabile e verificabile partendo da una struttura comune.

Data-driven execution, non dichiarativa. Il completamento di un elemento WBS non è un flag che un operatore alza cliccando "completato". È una condizione calcolata che richiede la verifica di più fonti: le quantità prodotte dall'impianto corrispondono a quelle del BIM? Il DDT è arrivato in cantiere? Il certificato di qualità è presente? Solo quando tutte le condizioni sono soddisfatte l'elemento è considerato certificato. Questo elimina la possibilità di gonfiare artificialmente i SAL.

Contract-aware delivery. L'avanzamento tecnico e l'avanzamento contrattuale sono trattati come due dimensioni dello stesso fenomeno, non come sistemi separati. Le milestone definite nel contratto (e arricchite dal Gantt) sono le condizioni che attivano i pagamenti. Il sistema non può certificare un SAL senza verificare le condizioni contrattuali associate.

Extensibility by design. Il sistema è progettato oggi per un singolo contratto per progetto, ma il modello dati è predisposto per la gestione multi-contratto (contratti padre/figlio, subappalti, dipendenze cross-contratto) senza richiedere migrazioni distruttive. `CONTRACT.parent_contract_id` e `MILESTONE.cross_contract_deps` esistono già nello schema anche se non ancora utilizzati.

Cloud Agnostic. Nessuna dipendenza da servizi proprietari di un singolo cloud provider. Ogni componente ha un'alternativa portatile: RedPanda al posto di Pub/Sub, MinIO al posto di S3 nativo, Keycloak al posto di Firebase Auth, Debezium al posto di Datastream. Il sistema gira identico su GCP, AWS, Azure o on-premise con un cambio di variabili d'ambiente.

Scelte Architetture: Perché e Come

CQRS — Command Query Responsibility Segregation

Il problema

In BuildTrust esistono due categorie di operazioni radicalmente diverse. Le operazioni di **scrittura** sono semplici per struttura ma eterogenee per origine: un parser BIM scrive 8 tipi di entità dal JSON Revit, un parser contratto scrive clausole e milestone estratte via LLM, un gateway OPC UA scrive record di produzione in streaming real-time, un operatore web scrive conferme di avanzamento. Ogni scrittura riguarda un dominio specifico e non richiede di "vedere" gli altri domini.

Le operazioni di **lettura** sono l'opposto: complesse per struttura ma omogenee per scopo. Una dashboard SAL deve aggregare dati da almeno 6 schemi diversi: quantità geometriche da `bim`, attività e milestone da `process`, voci economiche da `boq`, dati di produzione da `production`, avanzamenti da `progress`, certificati da `quality`. Un SAL Engine deve eseguire query su 15+ tabelle con join complessi per verificare l'incrocio quantità, la valorizzazione economica e la copertura documentale. Se write e read usano lo stesso modello di dati, queste query diventano lente (secondi, non millisecondi) e fragili.

La soluzione

CQRS separa nettamente il modello di scrittura dal modello di lettura. Il **Command Side** (write) scrive direttamente negli schemi di dominio PostgreSQL attraverso i parser e le Hasura Mutations. Ogni parser è responsabile solo del proprio schema; non conosce gli altri. Il **Query Side** (read) usa un modello separato: lo `schema: read`, che contiene **proiezioni read-optimized** già denormalizzate per le query di dashboard. In pratica non si tratta di materialized view PostgreSQL aggiornate riga-per-riga, ma di tabelle di proiezione e viste SQL semplici costruite sopra di esse. Queste proiezioni sono mantenute aggiornate da un projector Python che consuma eventi dal message broker. Le query di dashboard leggono solo da questo schema, ottenendo risposte in millisecondi invece di secondi.

La sincronizzazione tra write e read avviene attraverso il Change Data Capture (CDC): ogni INSERT o UPDATE nel database genera automaticamente un evento che il projector usa per aggiornare in modo incrementale le proiezioni read-side con `UPSERT` idempotenti. Questo garantisce che il Read Model sia sempre consistente con il Command Model, con un ritardo massimo di pochi secondi.

Perché non un approccio più semplice

L'alternativa sarebbe usare un unico modello normalizzato e costruire le query di lettura con join real-time. Questo funziona bene per sistemi con pochi dati e poche relazioni. Per BuildTrust, dove il SAL Engine deve incrociare dati da 6 domini su centinaia di elementi WBS, la performance degrada rapidamente. Con CQRS, il costo delle join complesse è pagato una sola volta (quando il projector aggiorna le proiezioni read-side) e non ad ogni query utente.

Event-Driven Architecture e CDC con Debezium

Il problema dell'accoppiamento

Senza un sistema di eventi, ogni componente del sistema dovrebbe chiamare direttamente gli altri. Quando il Production Ingestion Service registra un nuovo batch, dovrebbe chiamare il SAL Engine per aggiornare l'avanzamento, chiamare il Read Projector per aggiornare le dashboard, chiamare il Reconciler per verificare la copertura economica. Se uno di questi servizi è temporaneamente non disponibile, l'intera catena si rompe. Aggiungere un nuovo consumer (es. un sistema di notifiche) richiederebbe modificare il codice del Production Ingestion Service. Il sistema diventa un monolite distribuito.

Change Data Capture: perché il WAL e non eventi applicativi

La soluzione standard a questo problema sarebbe far emettere eventi direttamente dai servizi applicativi (pattern outbox o event sourcing). BuildTrust usa invece il **Change Data Capture** tramite Debezium, che legge il Write-Ahead

Log (WAL) di PostgreSQL e pubblica eventi per ogni modifica al database, indipendentemente da quale servizio l'ha causata.

Il vantaggio fondamentale del CDC è la **garanzia di consistenza**: un evento viene pubblicato solo se la modifica al database è stata effettivamente persisted. Con gli eventi applicativi, esiste sempre il rischio che l'applicazione emetta un evento ma poi il database fallisca il commit (o viceversa), creando inconsistenze. Con il CDC dal WAL, l'evento e il dato sono atomicamente legati: se il dato è nel WAL, l'evento sarà emesso; se il database non ha confermato il dato, l'evento non verrà mai emesso.

Un secondo vantaggio è la **retrocompatibilità**: è possibile aggiungere un nuovo consumer senza toccare il codice dei servizi esistenti. Basta registrare un nuovo subscriber sul topic RedPanda corrispondente. Questo è esattamente il caso dell'Audit Trail Consumer e del WBS-BOQ Reconciler, aggiunti in una fase successiva senza modificare i parser o il SAL Engine.

Debezium produce eventi strutturati con i valori `before` e `after` di ogni campo modificato, il timestamp della modifica e il tipo di operazione (INSERT/UPDATE/DELETE). Questo permette al projector di applicare le modifiche al Read Model in modo incrementale invece di ricalcolare tutto da zero.

RedPanda come Event Broker

Perché non Apache Kafka. Kafka è lo standard de facto per i sistemi event-driven ad alta scala, ma ha un costo operativo significativo: richiede ZooKeeper (o KRaft nelle versioni recenti), la JVM con le sue esigenze di tuning GC, e un cluster di almeno 3 broker per la tolleranza ai guasti. Per BuildTrust, che gestisce un volume di eventi moderato (qualche centinaio di eventi al minuto anche nel caso di picco di produzione), questo overhead è ingiustificato.

RedPanda è un broker Kafka-compatibile scritto in C++ (niente JVM, niente ZooKeeper) con un singolo binario. È completamente compatibile con il protocollo Kafka: Debezium, i consumer Python e qualsiasi libreria Kafka esistente funzionano senza modifiche. I vantaggi concreti per BuildTrust sono: latenza p99 inferiore a 10ms (contro i 50-100ms di Kafka), consumo di memoria 3-5x inferiore, nessun tuning GC, deploy con un singolo container invece di un cluster. In caso di necessità di scalabilità futura, la migrazione a Kafka è trasparente per tutto il codice applicativo perché l'interfaccia è identica.

Perché non Google Pub/Sub, AWS SQS/SNS o Azure Service Bus. Questi servizi managed sono convenienti su un singolo cloud provider ma violano il principio Cloud Agnostic. RedPanda gira identicamente on-premise, su GKE, su EKS o su AKS.

Architettura Esagonale (Ports & Adapters)

Il problema del lock-in tecnologico

Un microservizio che dipende direttamente da `google.cloud.storage` per salvare i file non può girare on-premise senza modifiche al codice. Un servizio che chiama `anthropic.Anthropic()` non può essere sostituito con OpenAI o con un modello locale Ollama senza refactoring. Un servizio che pubblica su RedPanda non può essere spostato su Kafka senza toccare la business logic. In un sistema progettato per essere cloud agnostic, questo tipo di accoppiamento è inaccettabile.

La soluzione: Ports & Adapters

L'architettura esagonale risolve questo problema strutturando ogni microservizio in tre strati separati. Il **domain layer** (`domain/`) contiene la business logic pura: come si parsano le clausole contrattuali, come si calcola il valore di un SAL, come si aggregano i batch di produzione. Questo strato non importa nessuna libreria esterna; usa solo interfacce astratte (le *ports*). Il **ports layer** (`ports/`) definisce le interfacce per tutte le dipendenze esterne: `StoragePort` con metodi `upload(file)` e `download(path)`, `EventPort` con `publish(topic, event)`, `LLMPort` con `extract_clauses(text)`. Il **adapters layer** (`adapters/`) contiene le implementazioni concrete di queste interfacce: `GCSAdapter` implementa `StoragePort` usando `google.cloud.storage`, `MinIOAdapter` implementa la stessa interfaccia usando `boto3`, `RedPandaAdapter` implementa `EventPort` usando `kafka-python`.

La selezione dell'adapter avviene a runtime tramite variabili d'ambiente:

```
STORAGE=gcs      → usa GCSAdapter
STORAGE=minio    → usa MinIOAdapter
STORAGE=local    → usa LocalFileAdapter (per sviluppo locale)

LLM=anthropic    → usa AnthropicAdapter
LLM=openai       → usa OpenAIAdapter
LLM=ollama       → usa OllamaAdapter (modello locale, zero costo, massima privacy)
```

Il risultato pratico è che i test unitari della business logic usano mock adapter senza nessuna dipendenza da servizi cloud. L'ambiente di sviluppo usa MinIO locale e RedPanda locale. La produzione usa GCS e RedPanda su Kubernetes. Nessun codice cambia tra ambienti: solo le variabili d'ambiente.

PostgreSQL: Single Database, Multi-Schema, Row Level Security

Perché non un database per microservizio

Il pattern standard dei microservizi prevede un database separato per ogni servizio. Questa scelta ha senso quando i servizi sono realmente indipendenti e non hanno bisogno di query cross-servizio. In BuildTrust, il SAL Engine ha un requisito preciso: deve leggere in una singola operazione transazionale dati da 6 domini diversi per eseguire l'incrocio quantità (BIM ↔ Production ↔ BOQ), la valorizzazione economica (BOQ × ELEMENT_PROGRESS) e la verifica documentale (QUALITY_TEST_CERT × WORK_PROGRESS).

Con database separati, questa operazione richiederebbe chiamate HTTP tra servizi o query federate, con problemi di consistenza transazionale, latenza aggiuntiva e complessità di error handling. Il SAL Engine non può "fermare il mondo" mentre raccoglie dati da 6 servizi REST: ha bisogno di una vista coerente dei dati in un singolo momento.

Un singolo database PostgreSQL con multi-schema risolve questo problema: il SAL Engine può eseguire join cross-schema in una singola transazione, con performance ottimali e garanzie ACID. Ogni microservizio scrive nel proprio schema (`bim`, `process`, `boq`, ecc.) senza mai toccare gli schema degli altri; solo il SAL Engine ha permessi di lettura cross-schema, e solo il Read Projector ha permessi di scrittura sullo `schema: read`.

Row Level Security per il multi-tenancy

PostgreSQL RLS permette di definire policy a livello di tabella che filtrano automaticamente le righe in base al tenant operativo corrente. In BuildTrust, ogni tabella ha una colonna `tenant_id` e una policy che forza il filtro: `WHERE tenant_id = current_setting('app.tenant_id')`. Quando la richiesta passa da Hasura, il valore deriva da `x-hasura-tenant-id`, che a sua volta è estratto dal JWT Keycloak. Quando invece la richiesta arriva da un microservizio interno (SAL Engine, projector, consumer, gateway OPC UA, MS-11), il servizio deve aprire la transazione impostando esplicitamente `SET LOCAL app.tenant_id = '<tenant>'` prima di leggere o scrivere i dati applicativi.

Il risultato è che anche se un bug applicativo dimenticasse di aggiungere il filtro tenant nella query, PostgreSQL lo applicherebbe comunque a livello di storage. Il RLS è quindi un safety net indipendente dal layer applicativo, ma funziona solo se **tutte** le connessioni applicative usano ruoli senza `BYPASSRLS` e impostano il tenant context all'inizio della transazione. Restano esclusi solo i ruoli DBA/amministrativi usati per manutenzione e migrazioni.

Hasura GraphQL Engine

Perché non un API layer custom

In un sistema con 21 entità distribuite su 8 schemi, costruire un layer API REST o GraphQL custom richiederebbe scrivere e mantenere CRUD per ogni entità, gestire la validazione dei tipi, implementare le Subscriptions WebSocket per gli aggiornamenti real-time, integrare la validazione JWT, gestire la paginazione e il filtering. La stima conservativa è 3-4 mesi di sviluppo solo per questo layer, che aggiunge valore zero alla business logic.

Hasura genera automaticamente l'intera API GraphQL a partire dallo schema PostgreSQL. Ogni tabella diventa immediatamente accessibile con Query, Mutation e Subscription (WebSocket) complete, con filtering, paginazione,

aggregazioni e relazioni. La validazione JWT di Keycloak è configurata direttamente in Hasura che estrae il `tenant_id` dal token e lo propaga come session variable alle policy RLS di PostgreSQL.

Hasura svolge un doppio ruolo nel sistema: è il **gateway di scrittura** per l'input manuale dell'operatore (avanzamento, qualità) e il **gateway di lettura** per le dashboard (legge dallo `schema: read`). Gli Event Triggers di Hasura permettono di chiamare il SAL Engine in modo sincrono ogni volta che viene inserita o aggiornata una riga in `WORK_PROGRESS`, `ELEMENT_PROGRESS`, `NON_CONFORMITY` o `QUALITY_TEST_CERT`, garantendo che il calcolo SAL sia riallineato anche quando cambia lo stato di dati già esistenti.

L'alternativa a Hasura sarebbe una soluzione custom in FastAPI o NestJS. Il vantaggio della soluzione custom sarebbe maggiore controllo; lo svantaggio è il tempo di sviluppo e la manutenzione continuativa. Per un sistema in cui la complessità è nella business logic (SAL Engine, parser, produzione) e non nel layer CRUD, Hasura è la scelta giusta.

Keycloak per Identity & Access Management

Perché non Auth0, Firebase Auth o Cognito

Auth0, Firebase Auth e Cognito sono servizi managed eccellenti, ma presentano tre problemi per BuildTrust. Primo, il **costo per utente**: in un modello SaaS B2B con decine o centinaia di utenti per cliente, i costi possono diventare significativi. Keycloak è open source e gira su container senza costi di licenza. Secondo, il **lock-in cloud**: Firebase Auth è legato a GCP, Cognito ad AWS. Keycloak gira ovunque. Terzo, la **portabilità on-premise**: molti clienti del settore costruzioni (grandi committenti, PA) richiedono che il sistema Identity giri nella loro infrastruttura. Keycloak supporta questo scenario nativamente.

Come funziona l'isolamento multi-tenant

Keycloak supporta il concetto di **Realm**: un realm è uno spazio di autenticazione completamente isolato con i propri utenti, ruoli, client e policy. In BuildTrust, ogni compagnia cliente ottiene il proprio realm in Keycloak. Gli utenti di Compagnia A non possono mai autenticarsi nel realm di Compagnia B; i token JWT sono emessi per un realm specifico e contengono il `tenant_id` come custom claim.

Hasura legge il `tenant_id` dal JWT e lo propaga come `x-hasura-tenant-id` in ogni query. PostgreSQL RLS usa questo valore per filtrare le righe. Si crea così un sistema a tre livelli di isolamento completamente indipendenti: se il layer applicativo fallisce, PostgreSQL mantiene l'isolamento; se PostgreSQL viene interrogato direttamente con credenziali errate, Keycloak ha già bloccato l'accesso.

I ruoli definiti in Keycloak (`admin`, `project_manager`, `dl`, `site_supervisor`, `quality_manager`) vengono inclusi nel JWT e usati da Hasura per applicare permission rules: un `site_supervisor` può inserire dati di avanzamento e qualità ma non può accedere alle dashboard SAL o rilasciare un SAL.

LLM Service come Gateway AI Unificato

Il problema dell'AI distribuita

Due microservizi usano capacità AI: il Contract Parser (MS-02) usa un LLM per estrarre clausole contrattuali da PDF, e il BOQ Parser (MS-03) usa un LLM per mappare le voci di computo alle attività WBS. Se ognuno di questi servizi gestisse direttamente la chiamata all'LLM, si creerebbero duplicazioni di codice, impossibilità di centralizzare il rate limiting e il caching, e dipendenza diretta da un singolo provider.

La soluzione: stateless gateway

MS-07 LLM Service è un microservizio Python **stateless** che espone un'API REST interna al cluster con capability specifiche: `extract_clauses(pdf_text)`, `map_boq_to_activities(boq_items, activities)`, `fuzzy_match(query, candidates)`. I chiamanti non sanno quale provider LLM è in uso; MS-07 gestisce il routing, il retry e il fallback.

L'adapter configurabile (`LLM=anthropic|openai|gemini|ollama`) permette tre scenari operativi: in produzione standard si usa Claude o GPT-4 per la massima qualità di estrazione; in ambienti con requisiti di privacy elevata si

usa Ollama con un modello locale (Mistral, LLaMA) che non invia mai dati fuori dal cluster; in sviluppo e test si usa un mock adapter che restituisce risposte predefinite senza costi.

Il caching delle risposte LLM è gestito centralmente in MS-07: se lo stesso testo contrattuale viene riprocessato (es. in caso di re-import), la risposta viene servita dalla cache senza nuovi costi API. Il rate limiting verso i provider esterni è applicato uniformemente.

Architettura di Sistema: Vista d'Insieme

I Sette Layer

L'architettura è organizzata in sette layer logici che riflettono il flusso dei dati dal punto di ingresso (utente o sistema esterno) fino al punto di fruizione (dashboard, SAL, notifiche).

Layer 1 — Presentation & Identity. Il frontend React SPA comunica con Keycloak per l'autenticazione OIDC. Al login, Keycloak emette un JWT firmato che contiene `tenant_id`, `user_id` e `roles`. Questo token viene allegato ad ogni richiesta verso Hasura.

Layer 2 — API Gateway. Hasura riceve le richieste GraphQL dal frontend, valida il JWT e applica le permission rules basate sui ruoli. Per le Mutations (scrittura manuale), scrive direttamente nel database. Per le Subscriptions, mantiene connessioni WebSocket con il frontend per inviare aggiornamenti real-time. Gli Event Triggers attivano MS-06 SAL Engine in modo sincrono su specifici eventi di insert e update.

Layer 3 — Write Side. I Command Services ricevono dati dalle tre sorgenti di ingestion: file dall'Object Storage (via Bucket Watcher), stream OPC UA dall'impianto, input manuale via Hasura. Ogni parser è responsabile di un singolo dominio e scrive solo nel proprio schema PostgreSQL.

Layer 4 — Persistence. PostgreSQL con 8 schemi core di dominio (`bim`, `process`, `boq`, `production`, `progress`, `quality`, `tenant`, `document`), più `read` per le proiezioni di lettura e `external` quando sono abilitate integrazioni di terze parti. Il SAL Engine (MS-06) è l'unico servizio applicativo con permessi di lettura cross-schema sui domini core; gli altri parser scrivono solo nel proprio schema.

Layer 5 — Event Pipeline. Debezium legge continuamente il WAL di PostgreSQL e pubblica su RedPanda un evento per ogni modifica. I topic sono separati per dominio (`bim.changes`, `production.changes`, ecc.) per permettere ai consumer di sottoscrivere solo i domini di loro interesse.

Layer 6 — Event Consumers & Read Side. I consumer Python sottoscrivono i topic RedPanda di interesse e reagiscono agli eventi: MS-09 aggiorna le proiezioni read-side per le dashboard, MS-10 invia notifiche, il Reconciler verifica la copertura economica, l'ACDAT Sync Consumer aggiorna il CDE esterno.

Layer 7 — External Integrations. MS-11 External Sync Service gestisce la sincronizzazione bidirezionale con piattaforme esterne (Procore, ACC, TeamSystem, Primavera). ACDAT (il CDE ufficiale italiano) è integrato tramite MS-08 ACDAT Connector e l'ACDAT Sync Consumer.

I Tre Canali di Ingestion

Canale 1: File Upload → Bucket Watcher → Parser

L'Object Storage (GCS in produzione su GCP, MinIO on-premise) è organizzato in bucket dedicati per tipo di file: `bim/`, `contratti/`, `computi/`, `gantt/`, `documenti/`. Quando un file viene caricato (dall'utente via UI o da un sistema esterno), il Bucket Watcher riceve una notifica (GCS Object Notifications o polling MinIO) e instrada il file al parser corretto basandosi sul bucket di destinazione e sull'estensione.

Questa architettura disaccoppiata ha un vantaggio importante: il caricamento di un file BIM da 50MB non blocca l'utente. Il parsing avviene in modo asincrono; l'utente può vedere il progresso tramite Subscription WebSocket su Hasura. Se il parser fallisce (file corrotto, formato non supportato), il file rimane nell'Object Storage e può essere riprocessato; nessun dato va perso.

Canale 2: OPC UA → Production Ingestion

Il protocollo OPC UA è lo standard industriale per la comunicazione con PLC e sistemi di automazione. L'impianto del calcestruzzo espone un server OPC UA con nodi che rappresentano i batch di produzione (volume, mix design, ID autobetoniera, timestamp). L'OPC UA Gateway (basato su `asyncua`, libreria Python asincrona) si connette al server OPC UA, sottoscrive i nodi di interesse e riceve notifiche in real-time ogni volta che un valore cambia.

I dati OPC UA hanno una natura "push" (il server notifica il client), non "pull". L'asynqua gestisce la reconnection automatica in caso di perdita di connessione con l'impianto. Il Production Ingestion Service riceve i dati normalizzati dal Gateway, arricchisce ogni batch con il riferimento all'elemento WBS corrispondente (derivato dalla mappatura attività) e scrive in `schema: production`.

Canale 3: Input Manuale (Hasura Mutations)

L'operatore di cantiere (capocantiere) e il Direttore Lavori inseriscono dati direttamente dal frontend React tramite GraphQL Mutations su Hasura. Le due categorie di input manuale sono: le conferme di avanzamento (`WORK_PROGRESS`, `ELEMENT_PROGRESS`) — che attestano il completamento di una fase di lavoro o l'arrivo di materiale in cantiere — e i dati di qualità (`QUALITY_TEST_CERT`, `NON_CONFORMITY`) — che registrano i risultati delle prove di laboratorio e le non conformità rilevate.

Questi sono gli **unici input che richiedono un'azione attiva dell'operatore**. Tutto il resto — produzione calcestruzzo, parsing BIM, analisi contratto — è automatico. Questa scelta è deliberata: ridurre al minimo il carico cognitivo dell'operatore di cantiere, che deve solo confermare ciò che è già successo, non inserire dati da zero.

Struttura dei Microservizi: Python e FastAPI

Template Esagonale Comune

Ogni microservizio BuildTrust è un container Docker Python 3.12 con FastAPI, strutturato in modo identico secondo il pattern Ports & Adapters. La struttura di directory è standardizzata per tutti i servizi:

```
ms-<nome>/
├── domain/
│   ├── __init__.py
│   ├── models.py           # Dataclass/Pydantic pure, zero dipendenze esterne
│   ├── services.py         # Business logic: parse, validate, calculate
│   └── exceptions.py       # Eccezioni di dominio tipizzate
├── ports/
│   ├── __init__.py
│   ├── storage.py          # StoragePort: upload/download/list
│   ├── events.py           # EventPort: publish(topic, payload)
│   ├── db.py               # DBPort: upsert, query, transaction
│   └── llm.py               # LLMPort: extract/map/match (solo MS-02/03/04/07)
├── adapters/
│   ├── storage/
│   │   ├── gcs.py          # GCSAdapter → google.cloud.storage
│   │   ├── s3.py           # S3Adapter → boto3
│   │   └── minio.py        # MinIOAdapter → boto3 endpoint_url
│   ├── events/
│   │   ├── redpanda.py     # RedPandaAdapter → confluent-kafka
│   │   └── kafka.py        # KafkaAdapter → confluent-kafka (stesso codice, diversa config)
│   ├── llm/
│   │   ├── anthropic.py    # AnthropicAdapter → anthropic SDK
│   │   ├── openai.py       # OpenAIAdapter → openai SDK
│   │   └── ollama.py        # OllamaAdapter → httpx POST /api/generate
│   └── db/
│       └── postgres.py     # SQLAlchemyAdapter → asyncpg
├── api/
│   ├── __init__.py
│   ├── router.py           # FastAPI router con tutti gli endpoint
│   ├── deps.py             # Dependency injection: get_db, get_storage, get_events
│   └── schemas.py          # Pydantic request/response models
├── config.py               # Pydantic Settings: legge ENV vars, seleziona adapter
├── main.py                 # FastAPI app, lifespan, health check
├── Dockerfile
└── pyproject.toml
```

La selezione dell'adapter avviene in `config.py` tramite `pydantic-settings` che legge le variabili d'ambiente:

```
# config.py
from pydantic_settings import BaseSettings
from typing import Literal

class Settings(BaseSettings):
    storage_backend: Literal["gcs", "s3", "minio"] = "minio"
    events_backend: Literal["redpanda", "kafka"] = "redpanda"
    llm_backend: Literal["anthropic", "openai", "ollama"] = "anthropic"
    database_url: str # postgresql+asyncpg://...

    class Config:
        env_file = ".env"

def get_storage_adapter(settings: Settings) -> StoragePort:
    match settings.storage_backend:
        case "gcs": return GCSAdapter()
        case "s3": return S3Adapter()
        case "minio": return MinIOAdapter()
```

Il `main.py` usa il lifespan di FastAPI per inizializzare le dipendenze una sola volta al boot:

```
# main.py
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    settings = Settings()
    app.state.storage = get_storage_adapter(settings)
    app.state.events = get_events_adapter(settings)
    app.state.db = await get_db_adapter(settings)
    yield
    await app.state.db.close()

app = FastAPI(title="ms-bim-parser", lifespan=lifespan)
app.include_router(router)

@app.get("/health")
async def health(): return {"status": "ok"}
```

MS-01 · BIM Parser

Trigger: evento da Bucket Watcher su topic `bucket.bim.uploaded` → `POST /parse`

Entità scritte: `BIM_MODEL`, `BIM_ELEMENT`, `BIM_QUANTITY`, `CONSTRUCTION_PHASE`, `WORK_ACTIVITY`, `MEASUREMENT_RULE`, `TIME_PROFILE`, `ELEMENT_ACTIVITY` (8 entità in una singola transazione)

Endpoints FastAPI:

```
POST /parse          → riceve {bucket, key, tenant_id}, avvia parsing asincrono
GET  /status/{job}   → stato del job (queued / parsing / done / failed)
POST /validate       → dry-run: valida il file JSON senza persistere
GET  /health
```

Domain logic chiave (`domain/services.py`):

Il parser legge il JSON Revit (fino a 50 MB) in streaming e costruisce un grafo di dipendenze tra le 8 entità. La transazione PostgreSQL è atomica: o tutte le 8 entità vengono inserite o nessuna. Usa `INSERT ... ON CONFLICT DO UPDATE` (upsert) per permettere il re-import senza duplicati. Al termine emette l'evento `bim.imported` con un payload di riepilogo: numero elementi, quantità aggregate per tipo, fasi trovate.

Struttura directory specifica:

```
ms-bim-parser/
├── domain/
│   ├── models.py      # BimModel, BimElement, BimQuantity, WorkActivity...
│   ├── services.py    # BimParserService.parse(json_path) → ParseResult
│   └── validators.py  # Schema JSON validation, IFC GUID format check
```

MS-02 · Contract Parser

Trigger: `bucket.contracts.uploaded` → `POST /parse`

Entità scritte: `CONTRACT`, `CLAUSE`, `PAYMENT_CONDITION`, `MILESTONE`

Endpoints FastAPI:

```
POST /parse          → parsing PDF/DOCX, chiama MS-07 per extraction
POST /validate       → preview clausole estratte senza persistere (Fase 5 UI)
POST /confirm/{id}   → l'utente ha validato: persiste le clausole e le condizioni
GET  /clauses/{contract_id} → lista clausole estratte con status (validated/pending)
GET  /health
```


Domain logic chiave:

Il parser usa `PyMuPDF` per estrarre testo da PDF e `python-docx` per DOCX. Il testo viene inviato a MS-07 LLM Service tramite chiamata HTTP interna al cluster. La risposta LLM è un JSON strutturato con le clausole categorizzate. Le clausole vengono presentate all'utente per validazione (Fase 5): **nessun dato viene persistito fino a `POST /confirm`**. Questa scelta elimina il rischio di dati contrattuali errati nel sistema.

MS-03 · BOQ Parser

Trigger: `bucket.boq.uploaded` → `POST /parse`

Entità scritte: `BOQ`, `BOQ_ITEM`, `BOQ_ACTIVITY`

Endpoints FastAPI:

```
POST /parse          → parsing XLSX computo
POST /map-activities → chiama MS-07 per matching BOQ ↔ WORK_ACTIVITY
POST /confirm/{id}   → persiste il mapping validato dall'utente
GET  /coverage/{project} → report copertura: WORK_ACTIVITY senza BOQ_ACTIVITY
GET  /health
```

Domain logic chiave:

Il parsing XLSX usa `openpyxl`. Il matching `BOQ_ACTIVITY` chiama MS-07 che restituisce per ogni voce di computo la `work_activity_id` proposta con un `confidence_score` (0.0–1.0). Colori nel frontend: > 0.8 verde (auto-accettato), 0.5–0.8 giallo (richiede conferma), < 0.5 rosso (deve essere mappato manualmente). La copertura economica è verificata dal WBS-BOQ Reconciler che consuma `boq.imported` su RedPanda.

MS-04 · Gantt Parser

Trigger: `bucket.gantt.uploaded` → `POST /parse`

Entità scritte: `GANTT_ACTIVITY`, arricchimento `MILESTONE`

Endpoints FastAPI:

```
POST /parse          → parsing MPP (via mpxj Java bridge) o XLSX
GET  /activities/{project} → attività Gantt con stato matching WBS
POST /link-activities → associa manualmente GANTT_ACTIVITY ↔ WORK_ACTIVITY
GET  /health
```

Domain logic chiave:

Il formato MPP (MS Project) viene letto tramite `mpxj`, una libreria Java che viene invocata tramite `subprocess` con un thin Python wrapper. Questo è il motivo per cui MS-04 ha un resource request più alto in Kubernetes (512 MiB): il processo Java viene avviato per ogni parsing e richiede più memoria. Per XLSX usa `openpyxl` direttamente. Il matching con le `WORK_ACTIVITY` esistenti avviene per codice (match esatto) o tramite MS-07 (fuzzy match semantico).

MS-05 · Production Ingestion

Trigger: OPC UA subscription + MQTT push + `bucket.production.uploaded` (batch offline)

Entità scritte: `PRODUCTION_RECORD`, `MIX_RECIPE`, `MIX_RECIPE_COMPONENT`

Endpoints FastAPI:

```
POST /record          → inserimento singolo record di produzione (da OPC UA gateway)
POST /batch           → inserimento batch di record (da upload file offline)
GET  /records/{project} → lista record con filtri (data, mix, elemento)
```

```
POST /recipes          → import ricette di mix design
GET  /health
```

Domain logic chiave (`domain/services.py`):

Il componente più critico per la latenza. Il gateway OPC UA (`asyncua`) gira in un loop asincrono separato e inserisce record nel database con `asyncpg` senza passare da FastAPI (per minimizzare la latenza). FastAPI espone solo l'API di ingestione manuale e il `POST /record` per i gateway che non supportano OPC UA diretto. Ogni `PRODUCTION_RECORD` viene associato automaticamente all'`ELEMENT_PROGRESS` tramite la mappatura `WORK_ACTIVITY → ELEMENT_ACTIVITY → BIM_ELEMENT`.

MS-06 · SAL Engine

Trigger: Hasura Event Trigger su INSERT/UPDATE in `WORK_PROGRESS`, `ELEMENT_PROGRESS`, `NON_CONFORMITY`, `QUALITY_TEST_CERT`

Entità scritte: `SAL`, `SAL_LINE_ITEM`, `SAL_CHECK_RESULT` (scrive su `schema: progress`)

Endpoints FastAPI:

```
POST /recalculate      → evento da Hasura su WORK_PROGRESS/ELEMENT_PROGRESS
POST /block-check      → evento da Hasura su NON_CONFORMITY
POST /verify-docs      → evento da Hasura su QUALITY_TEST_CERT
POST /release/{sal_id} → rilascio esplicito SAL da parte del DL
GET  /status/{project} → stato corrente SAL con dettaglio 6 check
GET  /history/{project} → lista SAL rilasciati con hash di integrità
GET  /health
```

Domain logic chiave:

Il SAL Engine usa SQLAlchemy con accesso in lettura a tutti gli schemi core applicativi. Ogni esecuzione apre una transazione impostando prima il contesto `app.tenant_id`, così le query cross-schema restano compatibili con RLS anche fuori da Hasura. La sessione SQLAlchemy è configurata con `schema_translate_map` per accedere a `bim.bim_element`, `production.production_record`, ecc. in modo trasparente. Ogni run dei 6 check scrive i risultati in `SAL_CHECK_RESULT`, permettendo di mostrare nel frontend lo stato parziale anche durante un calcolo in corso.

Il **release** (Passaggio 6) non salva solo uno stato finale ma un'evidenza certificabile. All'interno della stessa transazione il servizio: normalizza il payload di input, persiste uno snapshot immutabile dei dati usati per il rilascio, calcola l'hash SHA-256 deterministico dello snapshot, aggiorna `SAL.status = 'Released'` e scrive il riferimento all'hash. Se la transazione fallisce per qualsiasi motivo, il SAL resta in stato `Pending` e può essere rilasciato di nuovo. In questo modo non ci si limita a rilevare che i dati sottostanti sono cambiati: si conserva anche la fotografia esatta di ciò che è stato certificato.

MS-07 · LLM Service

Trigger: chiamate HTTP sincrone da MS-02, MS-03, MS-04

Endpoints FastAPI:

```
POST /extract-clauses → {pdf_text} → [{clause_type, text, extracted_data}]
POST /map-boq-activities → {boq_items[], activities[]} → [{boq_item_id, activity_id, confidence}]
POST /fuzzy-match      → {query, candidates[]} → [{candidate_id, score, reasoning}]
GET  /health
```

Domain logic chiave:

Stateless: nessuna scrittura su DB. Il caching delle risposte usa Redis (`redis-py`) con chiave = SHA-256 del payload di input. La TTL è configurabile per tipo di task (clause extraction: 24h perché il testo contrattuale non cambia; fuzzy

match: 1h perché le attività cambiano). Il fallback tra provider è implementato con `tenacity`: se la prima chiamata all'adapter principale fallisce, ritenta 2 volte poi switcha al provider di fallback.

```
# domain/services.py - pattern fallback
@retry(stop=stop_after_attempt(2), wait=wait_exponential(min=1, max=4))
async def extract_clauses(self, text: str) -> list[Clause]:
    try:
        return await self.primary_llm.extract_clauses(text)
    except LLMProviderError:
        return await self.fallback_llm.extract_clauses(text)
```

MS-08 · ACDAT Connector

Trigger: consumer topic `document.synced` + polling schedulato (APScheduler)

Endpoints FastAPI:

```
POST /sync/push      → pusha riferimento documento verso ACDAT
POST /sync/pull      → fetch stati workflow da ACDAT → aggiorna DOCUMENT_REF
GET  /status         → stato ultima sync (timestamp, documenti sincronizzati, errori)
POST /webhook        → riceve notifiche push da ACDAT (se supportate)
GET  /health
```

Domain logic chiave:

Il connector usa l'adapter `ACDAT=rest|webdav|mock`. In produzione si usa REST (se il CDE espone API) o WebDAV (se usa filesystem condiviso). Il topic `document.synced` è emesso dal workflow documentale interno quando un riferimento documento è pronto per essere riallineato con ACDAT; il polling schedulato usa `APScheduler` configurato per girare ogni 15 minuti in orario lavorativo per recuperare anche eventuali cambi di stato avvenuti direttamente nel CDE. I riferimenti documenti (`DOCUMENT_REF`) vengono aggiornati con lo stato del workflow ACDAT: `Shared` → `Published` → `Archived`. Un documento in stato `Published` in ACDAT è considerato prova documentale valida per il SAL Engine.

MS-09 · Read Model Projector

Trigger: consumer Kafka di tutti i topic `*.changes` da Debezium + `sal.computed`

Endpoints FastAPI:

```
GET  /lag            → consumer lag per topic (diagnostica)
POST /rebuild/{view} → rebuild completo di una proiezione read-side (admin)
GET  /health
```

Domain logic chiave:

Il projector è un consumer Kafka long-running che gira in un thread separato dal server FastAPI (che serve solo gli endpoint di diagnostica). Per ogni evento ricevuto, esegue un `UPDATE` o `UPSERT` idempotente sulle tabelle di proiezione nello `schema: read`, sopra le quali possono esistere viste SQL semplici per esposizione a Hasura. Le proiezioni gestite includono: `read.project_summary` (KPI progetto per Fase 3), `read.element_status` (stato colorazione per Digital Twin Fase 7), `read.sal_current` (dati aggregati per SAL Engine Fase 9), `read.portfolio_kpi` (KPI cross-progetto per Fase 2).

La semantica di delivery è **at-least-once**: se il projector crasha, al riavvio rilegge gli eventi dall'ultimo offset commitato e ri-applica le modifiche (le operazioni di aggiornamento delle proiezioni sono idempotenti).

MS-10 · Notification Service

Trigger: consumer topic `sal.released`, `sal.computed` (solo stato Blocked), `quality.changes` (NC severity=Critical)

Endpoints FastAPI:

```
GET /log/{project}    → storico notifiche inviate
POST /test            → notifica di test (admin)
GET /health
```

Domain logic chiave:

Il servizio usa adapter per i canali di notifica: `EMAIL=smtp|sendgrid`, `PUSH=fcm|apns`, `WEBHOOK=generic`. Ogni evento di dominio mappato a un template di notifica con il ruolo destinatario: SAL rilasciato → email a DL e RUP; NC Critical aperta → push al quality manager; SAL bloccato per NC Major → email al DL. La deduplication è gestita tramite `NOTIFICATION_LOG`: se lo stesso evento è già stato notificato, l'invio viene saltato.

MS-11 · External Sync Service

Descritto in dettaglio nella sezione Integrazioni Esterne.

Struttura directory specifica:

```
ms-external-sync/
├── domain/
│   ├── sync_engine.py    # SyncEngine: orchestrazione polling + entity mapping
│   ├── entity_mapper.py  # Mapping ID BuildTrust ↔ ID piattaforma esterna
│   └── conflict.py       # ConflictResolver: last-write-wins / escalation
├── adapters/
│   └── external/
│       ├── procure.py    # ProcureAdapter: REST v2 + OAuth 2.0
│       ├── acc.py        # ACCAdapter: APS REST + 3-legged OAuth
│       ├── itwin.py      # iTwinAdapter: iTwin Platform API
│       ├── trimble.py    # TrimbleAdapter: REST + BCF
│       ├── planradar.py  # PlanRadarAdapter: REST v2 + API Key
│       ├── aconex.py     # AconexAdapter: REST + SCP OAuth
│       ├── teamsystem.py # TeamSystemAdapter: file pickup XLSX/CSV
│       └── primavera.py  # PrimaveraAdapter: REST + XER export
└── api/
    └── router.py         # webhook receiver endpoints per ogni piattaforma
```

Modello Dati: Otto Schemi Core di Dominio

Filosofia del modello dati

Il modello dati di BuildTrust è costruito intorno a tre principi: ogni dato è ancorato alla WBS (via `WORK_ACTIVITY`), ogni attività WBS ha copertura economica (via `BOQ_ACTIVITY`), e l'avanzamento è calcolato non dichiarato. Gli 8 schemi core PostgreSQL riflettono esattamente i 6 domini di business più il tenant management e il documento ACDAT. A questi si affiancano due schemi di supporto: `read` per le proiezioni CQRS di lettura e `external` per la configurazione e l'audit trail delle integrazioni opzionali.

Schema BIM — La Struttura Portante

Lo schema `bim` contiene la rappresentazione del modello BIM importato da Revit. `BIM_MODEL` registra i metadati del modello (versione IFC, tool di authoring, data di import). `BIM_ELEMENT` rappresenta ogni singolo elemento costruttivo con il suo `IfcGuid` (identificatore univoco IFC), categoria, tipo, fase di costruzione e coordinate. `BIM_QUANTITY` associa a ogni elemento le sue quantità geometriche (volume in m³, area in m², lunghezza in m, peso in kg) con l'unità di misura.

`CONSTRUCTION_PHASE`, `WORK_ACTIVITY`, `MEASUREMENT_RULE` e `TIME_PROFILE` definiscono la **struttura WBS**: le fasi di costruzione (Fondazioni, Struttura, Finiture), le attività associate a ogni fase (Getto calcestruzzo, Posa armature, Scavo), le regole di misurazione (come si calcola la quantità completata per questa attività) e i profili temporali (produttività attesa in m³/ora, importati dal Gantt). `ELEMENT_ACTIVITY` è la tabella di mapping multi-a-molti tra elementi BIM e attività WBS: un diaframma può essere associato a tre attività (scavo, posa gabbia, getto) e un'attività può coprire più elementi.

Schema Process — Contratto e Milestone

Lo schema `process` contiene la struttura contrattuale estratta dal documento d'appalto. `CONTRACT` registra le parti, l'importo totale e il riferimento al documento originale. `CLAUSE` contiene ogni clausola estratta via LLM con categoria (Technical, Economic, Compliance, Payment), testo originale e struttura dati estratta. `PAYMENT_CONDITION` formalizza le condizioni di pagamento: soglia minima SAL (es. €750.000), formula di calcolo della rata (es. $OS \times RA_k / IC$), tempistica di emissione del certificato (7 giorni) e tempistica di pagamento (30 giorni). `MILESTONE` aggrega le milestone da due fonti: quelle estratte dal contratto (condizioni economiche) e quelle importate dal Gantt (tempistiche operative).

La separazione tra `CLAUSE` e `PAYMENT_CONDITION` è intenzionale: le clausole sono il testo grezzo estratto dall'LLM (con il relativo testo originale per verifica), le payment condition sono la struttura dati validata dall'utente che il SAL Engine userà per i calcoli. Questa doppia struttura permette di tracciare ogni condizione economica fino alla sua origine nel testo contrattuale.

Schema BOQ — Copertura Economica

Lo schema `boq` garantisce che ogni attività WBS abbia una valorizzazione economica. `BOQ` rappresenta il computo metrico importato (un file XLSX con intestazione progetto e versione). `BOQ_ITEM` contiene ogni voce di computo con il codice articolo (riferimento al prezzario regionale o aziendale), descrizione, unità di misura e prezzo unitario. `BOQ_ACTIVITY` è la tabella di mapping tra voci di computo e attività WBS, con un coefficiente per gestire i casi in cui una voce di computo copre parzialmente un'attività.

La **regola di copertura economica** è verificata automaticamente dal WBS-BOQ Reconciler ogni volta che cambia il modello BIM o il computo: ogni `WORK_ACTIVITY` deve avere almeno un `BOQ_ACTIVITY` associato. Se questa condizione non è soddisfatta, il SAL Engine blocca il calcolo per quella attività e segnala la lacuna all'utente.

Schema Production — I Dati dall'Impianto

Lo schema `production` registra ciò che effettivamente viene prodotto nell'impianto. `PRODUCTION_RECORD` è il record atomico di produzione: un batch di calcestruzzo con volume prodotto, timestamp di inizio e fine, ID autobetoniera, ricetta utilizzata, sorgente (OPC UA o inserimento manuale), esito del controllo qualità e riferimento all'`ELEMENT_PROGRESS` associato. `MIX_RECIPE` e `MIX_RECIPE_COMPONENT` definiscono le ricette di mix design usate: classe di resistenza, versione approvata, lista dei componenti con dosaggio per unità di volume.

Il collegamento tra `PRODUCTION_RECORD` ed `ELEMENT_PROGRESS` è il nodo critico del sistema: è il punto in cui i dati di produzione dell'impianto vengono associati a un elemento WBS del modello BIM. Questa associazione può avvenire automaticamente (se l'impianto trasmette l'ID elemento via OPC UA) o manualmente tramite la UI di "Progress from Field".

Schema Progress e SAL — Il Cuore del Business

Lo schema `progress` è lo schema più critico dal punto di vista del business. `WORK_PROGRESS` aggrega l'avanzamento di un'attività WBS in un periodo di tempo: data di inizio e fine, stato (NotStarted/InProgress/Certified/Blocked), certificazione del DL. `ELEMENT_PROGRESS` registra l'avanzamento del singolo elemento BIM: quantità completata, unità di misura, data, conferme (completamento fase, arrivo DDT, firma DL).

`SAL`, `SAL_LINE_ITEM` e `SAL_CHECK_RESULT` rappresentano il documento SAL vero e proprio. `SAL` contiene i valori aggregati (TPS, TCS, RA_k, stato), `SAL_LINE_ITEM` le voci per attività, `SAL_CHECK_RESULT` i risultati dei 6 check di verifica eseguiti dal SAL Engine prima del rilascio. Al momento del rilascio, `SAL` conserva anche uno snapshot immutabile dell'evidenza usata per il calcolo e il relativo hash di integrità. Questo schema è quindi **immutabile dopo il rilascio**: un SAL rilasciato non può essere modificato, solo revocato e ricreato.

Schema Quality — Verifica e Blocco

Lo schema `quality` contiene le prove e le non conformità. `QUALITY_TEST_CERT` registra ogni certificato di prova: tipo di test (compressione cls 28gg, trazione acciaio, controllo saldatura), risultato numerico, unità di misura, esito (PASS/FAIL/PENDING), laboratorio emittente e riferimento al `WORK_PROGRESS` validato. `NON_CONFORMITY` registra ogni difformità rilevata con severità (Minor/Major/Critical), descrizione, stato (Open/In Treatment/Closed) e — cruciale — il flag `blocks_sal`.

Una NC con `severity=Major` e `blocks_sal=true` impedisce il rilascio del SAL da parte del SAL Engine. Questa logica è implementata nel check 5 del SAL Engine ed è l'unico modo in cui il sistema blocca automaticamente un pagamento: non per una regola arbitraria ma per una non conformità documentata e tracciabile.

SAL Engine: Il Cuore Computazionale

Architettura del SAL Engine

MS-06 SAL Engine è il microservizio più complesso del sistema. Non è un semplice aggregatore: è un motore di calcolo che implementa la logica di certificazione dell'avanzamento in 6 passaggi sequenziali, ognuno dei quali può bloccare il processo se le condizioni non sono soddisfatte.

Il SAL Engine viene attivato da eventi Hasura su insert e update di `WORK_PROGRESS`, `ELEMENT_PROGRESS`, `NON_CONFORMITY` e `QUALITY_TEST_CERT`, oltre che dal comando esplicito di release. Questa scelta è intenzionale: in un dominio come il SAL non cambia solo "quando nasce un record", ma anche quando un certificato passa da `PENDING` a `PASS`, quando una NC viene chiusa o quando una quantità viene corretta. In tutti i casi, il SAL Engine legge i dati attuali da tutti i domini rilevanti, esegue i check e aggiorna lo stato del SAL corrente.

I Sei Passaggi di Validazione

Passaggio 1 — Parificazione automatica. Il SAL Engine aggrega tutte le quantità completate per ogni elemento WBS: somma i `PRODUCTION_RECORD` per gli elementi con dati OPC UA, legge le quantità confermate dagli `ELEMENT_PROGRESS` per gli elementi con input manuale, incrocia con le `BIM_QUANTITY` per avere le quantità totali di riferimento. Il risultato è una tabella di avanzamento per elemento WBS con percentuale di completamento.

Passaggio 2 — Incrocio quantità (BIM ↔ Production ↔ BOQ). Per ogni attività WBS, il SAL Engine verifica che le quantità da tre fonti siano coerenti: le quantità geometriche dal BIM (cosa dovrebbe essere), le quantità prodotte dall'impianto o confermate manualmente (cosa è stato fatto), le quantità del computo metrico (cosa è stato valorizzato). La discrepanza accettabile è configurabile (target ≥ 98%). Se la discrepanza supera la soglia, il check genera un warning o un blocco a seconda della configurazione.

Passaggio 3 — Valorizzazione economica. Per ogni `BOQ_ACTIVITY`, il SAL Engine calcola il valore economico proporzionale alla quantità completata: $\text{valore} = \text{BOQ_ITEM.unit_price} \times \text{BOQ_ACTIVITY.coeff} \times \text{ELEMENT_PROGRESS.qty_done}$. Somma tutti questi valori per ottenere il TPS (Totale Progressivo SAL). Verifica che il TPS superi la soglia contrattuale minima definita in `PAYMENT_CONDITION` prima di procedere. Calcola la rata di acconto `RA_k` usando la formula contrattuale.

Passaggio 4 — Verifica documentale. Il SAL Engine verifica che ogni `WORK_PROGRESS` certificato abbia la documentazione di supporto necessaria: almeno un `QUALITY_TEST_CERT` con outcome PASS per i materiali soggetti a certificazione (calcestruzzi, acciai), almeno un DDT confermato per i materiali arrivati in cantiere. La copertura documentale è calcolata come percentuale: target ≥ 98%.

Passaggio 5 — NC Check. Il SAL Engine interroga `NON_CONFORMITY` e verifica che non ci siano NC aperte con `severity=Major` e `blocks_sal=true` sugli elementi WBS inclusi nel SAL. Se trovate, il SAL viene bloccato e la ragione di blocco viene scritta in `SAL_CHECK_RESULT`.

Passaggio 6 — Rilascio SAL. Se tutti i check precedenti sono superati, il SAL Engine congela il SAL: marca lo stato come `Released`, salva il timestamp di rilascio, persiste uno snapshot normalizzato dei dati di input e firma lo snapshot con un hash deterministico. Questo hash serve come prova di integrità; lo snapshot, invece, serve come prova ricostruibile di ciò che è stato effettivamente certificato in quel momento.

Integrazioni Esterne

ACDAT: il CDE Ufficiale Italiano

L'ACDAT (Ambiente di Condivisione Dati) è il sistema documentale ufficiale previsto dalla normativa italiana per i progetti BIM (D.Lgs. 36/2023 e relative linee guida UNI EN ISO 19650). È il Common Data Environment (CDE) dove si sviluppa l'intero workflow autorizzativo: condivisione bozze, revisione, approvazione, pubblicazione e archiviazione dei documenti di progetto.

BuildTrust non sostituisce l'ACDAT. Il suo ruolo è diverso: usa i documenti archiviati nell'ACDAT come fonti di dati e come evidenza a supporto del SAL. La struttura delle cartelle ACDAT deve riflettere quella della WBS in modo che ogni documento sia tracciabile all'attività di progetto corrispondente.

L'integrazione avviene su due livelli: MS-08 ACDAT Connector gestisce la sincronizzazione bidirezionale con il CDE ACDAT (quando un documento viene approvato in ACDAT, il suo stato viene aggiornato in BuildTrust; quando un SAL viene rilasciato in BuildTrust, il riferimento viene archiviato in ACDAT). L'ACDAT Sync Consumer gestisce la sincronizzazione asincrona event-driven specifica verso ACDAT: quando vengono registrati nuovi `ELEMENT_PROGRESS` o `QUALITY_TEST_CERT`, il consumer aggiorna automaticamente i riferimenti documentali nel CDE ufficiale.

MS-11 External Sync Service: il Gateway verso l'Ecosistema

Il settore delle costruzioni usa decine di piattaforme diverse per BIM authoring, project management, PLM e document management. BuildTrust deve integrarsi con questo ecosistema senza diventare dipendente da nessuna piattaforma specifica. MS-11 External Sync Service è il componente che gestisce tutta la comunicazione bidirezionale con le piattaforme esterne.

I Quattro Cluster di Integrazione

Cluster	Piattaforme	Protocollo	Pattern
A — BIM Authoring	Revit, Civil 3D, Tekla Structures, OpenBuildings, Navisworks, CATIA	IFC 4.x export + APS / iTwin REST API (OAuth 2.0)	File-based (primario) - REST delta sync (futuro)
B — Cloud CDE	ACC / BIM 360, Trimble Connect, Bentley iTwin, ProjectWise	REST API + OAuth 2.0 + Webhooks + BCF topics	REST API bidirezionale real-time
C — Construction PM	Procore, PlanRadar, Oracle Aconex, TeamSystem CPM	REST API + Webhooks (TeamSystem: file XLSX/CSV)	REST/Webhooks + file import ibrido
D — PLM / Enterprise	Dassault 3DEXperience, Siemens Teamcenter, Oracle Primavera P6, MS Project	REST/SOAP + file exchange (XER, MPP)	Batch sync schedulato

Cluster A — BIM Authoring Tools

Tutti gli strumenti BIM esportano in formato IFC (4.0 / 4.3), già gestito dal BIM Parser (MS-01). Il flusso primario non cambia: file IFC caricato nell'Object Storage → Bucket Watcher → MS-01. L'integrazione MS-11 aggiunge la possibilità di sync incrementale per le piattaforme con API REST mature:

Piattaforma	API disponibile	Uso in BuildTrust
Autodesk Revit / Civil 3D	APS (Autodesk Platform Services) REST, OAuth 2.0	IFC primario; futuro: APS per delta sync e metadata extraction
Tekla Structures	IFC 4.3 + Trimble Connect API REST	IFC + Trimble Connect per BCF topic sync
Bentley OpenBuildings	iTwin Platform REST API (50+ formati)	IFC + iTwin API per digital twin sync e change tracking

Piattaforma	API disponibile	Uso in BuildTrust
Navisworks	IFC export + APS Viewer API (clash detection)	IFC import; possibile integrazione futura clash results
Dassault CATIA	3DEXperience REST API (3DSpace)	Batch sync BOM per scenari EPC

Cluster B — Cloud CDE

Le piattaforme CDE cloud offrono REST API mature con OAuth 2.0. L'integrazione è bidirezionale: BuildTrust legge (documenti, modelli, issues) e scrive (stati SAL, nuovi documenti).

Piattaforma	Auth	Dati sincronizzabili
ACC / BIM 360	OAuth 2.0 3-legged, Custom Integration in Account Admin	Documenti, Issues/RFI, modelli IFC, clash results, change orders
Trimble Connect	Trimble Identity OAuth	Modelli .tekla/.ifc, BCF topics, file e cartelle progetto
Bentley iTwin	OAuth 2.0 Azure-based	iModels, IoT sensor data, reality capture, change tracking
ProjectWise	Bentley IMS OAuth	Documenti, workflow approvativo, metadata

MS-11 registra un webhook receiver per ogni piattaforma. Quando un documento viene approvato in ACC, il webhook chiama `POST /webhook/acc` su MS-11, che crea un evento `external.cde.changes` su RedPanda. Un consumer dedicato dell'integrazione esterna aggiorna `DOCUMENT_REF` e `EXT_SYNC_LOG` in BuildTrust; solo se il progetto prevede anche ACDAT come CDE ufficiale, MS-08 può successivamente propagare o riallineare i riferimenti verso ACDAT.

Cluster C — Construction PM

Questo cluster ha la maggiore variabilità di maturità API.

Piattaforma	Integrazione	Note
Procore	REST v2 API, OAuth 2.0, Webhooks nativi	API più matura del settore. Sync bidirezionale: RFI, Daily Logs, Change Orders, Cost Codes
PlanRadar	REST API v2, API Key, Webhooks per ticket events	Forte per defect management. Rate limit 30 req/min
Oracle Aconex	REST API JSON/XML, SCP OAuth 2.0	Ruolo simile ad ACDAT. Documenti, mail, transmittals
TeamSystem CPM	Nessuna REST API pubblica per DL. Export XLSX/CSV	Integrazione file-based: pickup da SFTP o directory condivisa

Pattern ibrido per TeamSystem: MS-11 monitora una cartella SFTP configurata. Ogni file XLSX depositato da TeamSystem CPM viene rilevato dal `FilePickupAdapter`, parsato e convertito in eventi `external.teamsystem.changes`. L'utente deve solo configurare una job schedulata di export in TeamSystem e l'indirizzo SFTP. Nessun codice da scrivere nel CM.

Cluster D — PLM / Enterprise

Rilevanti per scenari EPC complessi (Engineering, Procurement, Construction):

- **Oracle Primavera P6:** REST API + export XER per schedule sync. Complementare al Gantt Parser MS-04.
- **Dassault 3DEXperience:** REST API (3DSpace) per lifecycle management e BOM. Pattern batch sync schedulato.
- **Siemens Teamcenter:** TC Gateway REST API. Integrazione con ENOVIA per lifecycle states.

- **MS Project:** File MPP già gestito da MS-04. Nessun nuovo adapter necessario.

Schema PostgreSQL `external`

MS-11 aggiunge un nuovo schema al database per tracciare la configurazione e l'audit trail di ogni integrazione:

Entità	Campi chiave	Scopo
<code>EXT_SYNC_CONFIG</code>	platform, endpoint_url, secret_ref, poll_interval_min, sync_direction, active	Configurazione per piattaforma: credenziali (riferimento a secret manager, mai in chiaro), frequenza polling, direzione sync
<code>EXT_SYNC_LOG</code>	timestamp, platform, direction (in/out), entity_type, bt_entity_id, ext_entity_id, outcome, error_detail	Audit trail immutabile di ogni operazione di sync
<code>EXT_ENTITY_MAP</code>	platform, bt_entity_id, ext_entity_id, entity_type, last_sync_at	Mapping bidirezionale tra ID BuildTrust e ID piattaforma esterna

I Tre Pattern di Integrazione

Pattern A — File-based (già operativo): IFC/XLSX → Object Storage → Bucket Watcher → Parser. Copre BIM Authoring, MS Project, TeamSystem (XLSX).

Pattern B — REST API sync (via MS-11): polling periodico o webhook receiver → entity mapping → eventi RedPanda. Il ciclo di vita di un'integrazione REST è: (1) MS-11 effettua il primo fetch completo e popola `EXT_ENTITY_MAP`; (2) i fetch successivi recuperano solo i delta (`modified_after=last_sync_at`); (3) ogni modifica genera un evento su RedPanda; (4) il consumer appropriato aggiorna il DB BuildTrust.

Pattern C — File pickup (via MS-11): MS-11 monitora directory SFTP o condivisa → parsing XLSX/CSV → domain entities. Specifico per piattaforme senza REST API pubbliche.

Adapter selezione via ENV

```
EXT_ADAPTERS=procore,acc,teamsystem
PROCORE_CLIENT_ID=xxx
PROCORE_CLIENT_SECRET=yyy
ACC_CLIENT_ID=xxx
ACC_CLIENT_SECRET=yyy
TEAMSYSTEM_SFTP_HOST=sftp.example.com
TEAMSYSTEM_SFTP_PATH=/export/buildtrust/
```

L'attivazione di un adapter è zero-code: si aggiunge la piattaforma a `EXT_ADAPTERS` e si configurano le credenziali. Il deployment è un semplice `kubectl rollout restart deployment/ms-external-sync`.

Sicurezza e Multi-Tenancy

I Tre Livelli di Isolamento

La sicurezza multi-tenant in BuildTrust è implementata a tre livelli indipendenti. L'indipendenza è il punto chiave: un bypass a un livello non compromette gli altri.

Livello 1 — Keycloak (Identity). Ogni tenant ha un realm separato in Keycloak. Un utente di Tenant A non può ottenere un token valido per Tenant B: i realm sono completamente isolati. Il JWT emesso da Keycloak contiene il `tenant_id` come custom claim firmato: non può essere modificato senza invalidare la firma del token.

Livello 2 — Hasura (Authorization). Hasura valida il JWT ad ogni request, estrae il `tenant_id` e lo imposta come session variable `x-hasura-tenant-id`. Le permission rules di Hasura verificano che ogni query includa un filtro sul `tenant_id` corrispondente. Un utente non può eseguire query su dati di un altro tenant anche se costruisce manualmente la query GraphQL, perché Hasura aggiunge sempre il filtro.

Livello 3 — PostgreSQL RLS (Data isolation). Le policy RLS su ogni tabella filtrano le righe in base a `current_setting('app.tenant_id')`. Quando la richiesta arriva da Hasura, il valore è impostato dal layer GraphQL; quando arriva da un microservizio interno, viene impostato esplicitamente dal servizio all'inizio della transazione. Anche se una query applicativa dimenticasse il filtro tenant, il database continuerebbe a vedere solo il tenant attivo per quella sessione. L'unica eccezione è rappresentata dai ruoli DBA/amministrativi usati fuori dal normale traffico applicativo.

Gestione dei Ruoli

I ruoli Keycloak si mappano su permission rules Hasura con granularità per operazione e per colonna. Questa separazione riduce la superficie di attacco e rispetta il principio del minimo privilegio.

Ruolo	Schemi leggibili	Scrittura	Azioni speciali
<code>admin</code>	Tutti	Tutti	Gestione tenant, configurazione sistema
<code>project_manager / rup</code>	Tutti	process, boq	Rilascio SAL, approvazione clausole
<code>dl</code> (Direttore Lavori)	Tutti	progress, quality	Rilascio SAL, certificazione WORK_PROGRESS
<code>site_supervisor</code>	bim, progress, production	progress, quality	Inserimento WORK_PROGRESS, ELEMENT_PROGRESS
<code>quality_manager</code>	quality, document, bim	quality	Inserimento QUALITY_TEST_CERT, NON_CONFORMITY

Un `site_supervisor` può inserire conferme di avanzamento e dati di qualità ma non vede `SAL`, `PAYMENT_CONDITION` o `BOQ_ITEM`: non ha accesso all'aspetto economico del progetto. Un `dl` può leggere tutto e rilasciare un SAL ma non può modificare BIM o contratto, domini di competenza esclusiva del project manager. La granularità per colonna in Hasura permette di nascondere, ad esempio, il prezzo unitario di un `BOQ_ITEM` al `site_supervisor` pur consentendogli di vedere la descrizione dell'attività.

Sicurezza del Canale OPC UA

I dati provenienti dagli impianti industriali tramite OPC UA non passano attraverso l'autenticazione Keycloak standard: il gateway OPC UA si autentica con un service account dedicato (credenziale machine-to-machine, non utente). Questo service account ha permessi di sola scrittura sullo schema `production` e nessun accesso agli altri schemi. Ogni record inserito eredita il `tenant_id` configurato nel servizio, non derivato da un JWT utente. Questa separazione garantisce che un eventuale compromesso del gateway OPC UA non possa accedere ai dati contrattuali o economici del sistema.

Flusso UI: Le 9 Fasi

Il frontend React segue un flusso strutturato in 9 fasi che riflette il ciclo di vita di un progetto infrastrutturale. Il design system usa un dark theme blu scuro/navy con glassmorphism, scelto per ridurre l'affaticamento visivo in ambienti di cantiere dove il monitor è spesso in condizioni di luce difficili.

Fasi 1–3 (Login, Portfolio, Scheda Progetto) costituiscono il punto di ingresso. La Fase 2 mostra il portfolio multi-progetto con KPI aggregati (valore totale, SAL rilasciati, avanzamento medio). La Fase 3 mostra il dettaglio di un singolo progetto con il rendering 3D degli elementi colorati per stato e la tabella SAL.

Fasi 4–6 (Importa BIM, Importa Contratto, Check Computo) sono la configurazione iniziale del progetto, la fase più critica. La Fase 4 importa il modello BIM via drag & drop del JSON Revit; l'anteprima mostra immediatamente il numero di elementi trovati, le quantità e le fasi identificate. La Fase 5 importa il contratto PDF/DOCX: MS-02 invia il testo al LLM Service, che estrae le clausole e le categorizza (Technical, Economic, Compliance, Payment); l'utente valida le clausole estratte prima di confermare. La Fase 6 verifica la corrispondenza WBS ↔ Computo: ogni attività WBS deve trovare corrispondenza nel computo metrico importato; le lacune sono evidenziate in rosso.

Fasi 7–8 (Digital Twin View, Progress from Field) sono le fasi operative quotidiane. La Fase 7 mostra la visualizzazione 3D del modello con gli elementi colorati per stato di avanzamento (blu = non iniziato, arancione = in corso, verde = certificato, rosso = bloccato) e una timeline SAL per navigare tra periodi di rendicontazione. La Fase 8 è il cuore dell'operatività di cantiere: la colonna sinistra mostra i dati real-time dall'impianto OPC UA (batch di produzione con timestamp, volume, stato), la colonna destra permette al DL di inserire le conferme manuali. Il sistema calcola automaticamente l'aggregazione delle quantità per elemento WBS.

Fase 9 (SAL Engine) è la fase di rendicontazione. Il SAL Engine mostra in tempo reale lo stato dei 6 check (verde = OK, giallo = warning, rosso = blocco). Il pulsante "Rilascia SAL" è attivo solo quando tutti i check sono verdi. Il rilascio è un'azione irreversibile che congela il SAL e genera il documento certificativo.

Riepilogo delle 9 Fasi

Fase	Nome	Attore principale	Dati in ingresso	Output
1	Login	Tutti	Credenziali Keycloak	JWT con tenant_id e ruolo
2	Portfolio Progetti	RUP / PM	Read Model (vista portfolio_kpi)	Dashboard KPI cross-progetto
3	Scheda Progetto	DL / RUP	Read Model (vista project_summary)	Dettaglio SAL, rendering 3D stato
4	Importa BIM	PM / DL	File JSON Revit (IFC4)	8 entità BIM persistite, evento bim.imported
5	Importa Contratto	PM / RUP	File PDF / DOCX contratto	Clausole estratte via LLM, validate dall'utente
6	Check Computo	PM / DL	File XLSX computo metrico	BOQ mappato su WBS, copertura economica verificata
7	Digital Twin View	DL / PM	Read Model (vista element_status)	3D colorato per stato, timeline SAL navigabile
8	Progress from Field	Capocantiere / DL	OPC UA stream + input manuale	WORK_PROGRESS, ELEMENT_PROGRESS → trigger SAL Engine
9	SAL Engine	DL / RUP	Cross-schema (6 check)	SAL rilasciato con hash integrità e documento certificativo

Principi di UX

Il frontend è progettato per ridurre al minimo il carico cognitivo degli operatori di cantiere. Le Fasi 4-6 (configurazione) sono eseguite una sola volta per progetto dal project manager. Le Fasi 7-8 (operatività) sono quelle di uso quotidiano e devono funzionare anche su tablet in condizioni di connettività ridotta. La Fase 9 (SAL) è quella con maggiore peso decisionale e per questo espone il massimo dettaglio sullo stato di ogni check, con motivazioni esplicite per ogni blocco.

Dimensionamento e Deploy

GCP come Target di Riferimento

GCP è la piattaforma cloud di riferimento per BuildTrust, ma l'architettura è identica su AWS o Azure con i soli adapter swap. Il deployment è però **ibrido per natura del workload**: i servizi HTTP stateless e request/response (parser, Hasura-adjacent API, LLM gateway, webhook receiver) possono girare su Cloud Run o su un equivalente serverless container; i componenti long-running o stateful (RedPanda, Debezium, projector, notification consumer, scheduler ACDAT, gateway OPC UA, eventuali poller di MS-11) devono invece girare su GKE Autopilot o su un cluster Kubernetes equivalente.

Lo scenario di dimensionamento di riferimento è: 5 progetti attivi in parallelo, 1.000–2.000 elementi BIM per progetto, 3–5 impianti OPC UA che inviano dati ogni 30 secondi, 10–20 utenti concorrenti. In questo scenario, il componente più esigente è PostgreSQL Cloud SQL, che deve gestire le query del SAL Engine e aggiornare il Read Model.

Componente	Sizing	Costo stimato GCP
Cloud Run (servizi stateless)	Auto-scaling, scale-to-zero	50–150 €/mese
GKE Autopilot (broker + worker long-running)	Auto-scaling	300–500 €/mese
Cloud SQL PostgreSQL	2 vCPU / 7.5 GB Enterprise	250–350 €/mese
GCS Object Storage	Pay-per-use	< 5 €/mese
LLM API	Dipendente dal volume di parsing	50–200 €/mese
Totale		650–1.200 €/mese

La scalabilità è pressoché lineare: raddoppiando il numero di progetti si raddoppia principalmente il costo Cloud SQL e, in seconda battuta, il footprint dei worker long-running su GKE. I servizi stateless su Cloud Run scalano automaticamente senza costi fissi rilevanti; i consumer e i gateway invece restano sempre attivi per definizione.

Nota sulla versione GCP-nativa. È possibile sostituire Bucket Watcher con Cloud Storage Notifications + Pub/Sub trigger, Debezium con Cloud Datastream, e RedPanda con Cloud Pub/Sub. Questo semplifica l'operatività eliminando componenti da gestire, ma introduce lock-in su GCP. La scelta tra portabilità e semplicità operativa è deliberata e deve essere valutata in base alla strategia del cliente.

Modello Dati: Esempio Completo

Per rendere concreto il modello dati, questo scenario traccia il ciclo di vita completo di un elemento costruttivo dalla progettazione alla certificazione del SAL.

Scenario: Opera Edificio Residenziale — Progetto Levante. Fase STR (Struttura). Lavorazione: Getto calcestruzzo platea di fondazione. Elemento BIM: plinto `E_FND_001` da 3.85 m³.

Il **modello BIM** importato da Revit (IFC4, tool Revit, ModelId M001, ProjectCode 0549-IDG) contiene l'elemento `E_FND_001` con categoria Fondazioni, famiglia Plinti, tipo "Plinto CLS", quota -1. Le quantità associate sono: Volume = 3.85 m³, Area = 6.20 m². Il BIM non contiene costi né tempi: fornisce solo le quantità geometriche verificate.

Il **processo** associa l'elemento a due attività WBS: `CLS-GET` (Getto calcestruzzo) e `CLS-POM` (Pompaggio calcestruzzo), entrambe nella fase STR. La `MEASUREMENT_RULE` per `CLS-GET` definisce che la quantità di riferimento è `E_BIM_QUANTITY.Volume` in m³. Il `TIME_PROFILE` stima una produttività di 25 m³/ora.

Il **computo metrico** (BOQ CME-0549 "Computo metrico Levante") contiene la voce `CLS-C25`: Calcestruzzo C25/30, m³, prezzo unitario 145,00 €. La `BOQ_ACTIVITY` lega questa voce all'attività `CLS-GET` con coefficiente 1.0. Valorizzazione: 3.85 m³ × 145,00 €/m³ = **558,25 €**.

La **produzione** avviene il 10 febbraio 2026. L'impianto OPC UA trasmette due batch: PR001 (1.40 m³, 08:42–09:10) e PR002 (2.45 m³, 10:30–12:00), entrambi con mix recipe `CLS-C25` (Cemento 320 kg/m³, Sabbia 780 kg/m³, Ghiaia 1050 kg/m³, Acqua 180 l/m³), entrambi con qualità OK. Totale prodotto: **3.85 m³**.

L'**avanzamento** viene confermato dal DL che crea `WP001` (start 08:00, end 18:00, certified=Yes) e `EP001` (ElementId=E_FND_001, qty_done=3.85 m³). Questo evento triggera il SAL Engine via Hasura Event Trigger.

La **qualità** è verificata da `QC001`: test di compressione a 28 giorni, risultato 32.5 MPa, esito PASS (C25/30 richiede min 25 MPa). NC001 (slump borderline, severity=Minor) è già in stato Closed e non blocca il SAL.

Il **SAL Engine** esegue i 6 check: quantità BIM (3.85 m³) = quantità prodotta (3.85 m³) = quantità computo (3.85 m³) → incrocio OK al 100%. Valorizzazione 558,25 €. DDT presente. Nessuna NC Major aperta. SAL rilasciabile.

Evoluzioni Future

Multi-contratto avanzato. Lo schema è già predisposto con `CONTRACT.parent_contract_id` e `MILESTONE.cross_contract_deps`. La fase implementativa richiederà di estendere il SAL Engine per calcolare SAL per sub-contratto e gestire le dipendenze tra milestone di contratti diversi (es. una milestone del sub-contratto di prefabbricazione che sblocca una milestone del contratto principale).

Workflow documentale evoluto. L'integrazione ACDAT attuale è bidirezionale ma non gestisce il workflow approvativo interno ad ACDAT (condivisione → revisione → approvazione → pubblicazione). Una versione futura potrebbe includere la gestione degli stati di workflow ACDAT direttamente da BuildTrust, eliminando la necessità di accedere separatamente al CDE per le azioni di approvazione documentale.

Analisi predittiva. Il sistema raccoglie già tutti i dati necessari per la previsione dei ritardi: produttività storica per tipo di attività, variazioni tra pianificato e consuntivo, correlazioni tra NC e ritardi. Un servizio di analisi predittiva potrebbe usare questi dati per segnalare in anticipo rischi di sfioramento delle milestone contrattuali.

Profilazione avanzata. La gestione dei permessi attuale è a livello di ruolo globale. Una versione futura permetterà permessi granulari per sub-lotto di progetto, visibilità parziale per i subappaltatori (che vedono solo le proprie attività), e accesso read-only per il committente (che vede l'avanzamento ma non i dettagli economici interni).

Gestione multi-contratto. Il modello attuale supporta un contratto principale per progetto. Una versione futura estenderà il SAL Engine per gestire contratti paralleli (principale + subappalti) con dipendenze cross-contratto: una milestone del subappaltatore che sblocca una milestone del contratto principale, o un SAL parziale calcolato solo sulle attività di un determinato subappaltatore.

Mobile-first per il cantiere. L'interfaccia delle Fasi 7 e 8 è pensata per desktop. Una versione mobile nativa (React Native o PWA offline-first) permetterebbe al capocantiere di inserire conferme di avanzamento direttamente dal cantiere, con sincronizzazione differita quando la connettività è disponibile. I dati inseriti offline verrebbero accodati localmente e sincronizzati al rientro in area di copertura.

Integrazione con droni e realtà aumentata. I dati di avanzamento potrebbero essere alimentati anche da scansioni fotogrammetriche (drone survey) che producono nuvole di punti georeferenziate. Un parser dedicato confronterebbe la geometria misurata con quella del modello BIM, calcolando automaticamente le quantità completate senza input manuale del capocantiere.

Glossario

Termine	Significato
WBS	Work Breakdown Structure — struttura gerarchica delle attività di progetto, backbone del sistema
SAL	Stato Avanzamento Lavori — documento certificativo per l'abilitazione dei pagamenti contrattuali
BOQ	Bill of Quantities — Computo Metrico, valorizzazione economica delle attività WBS
BIM	Building Information Modeling — modello digitale dell'opera con geometrie e quantità
IFC	Industry Foundation Classes — formato standard aperto per lo scambio di modelli BIM
DDT	Documento di Trasporto — bolla di accompagnamento materiali, evidenza di arrivo in cantiere
DL	Direttore Lavori — figura contrattuale responsabile della supervisione tecnica
RUP	Responsabile Unico del Procedimento — figura responsabile per gli appalti pubblici
OPC UA	Open Platform Communications Unified Architecture — protocollo standard industriale per PLC e automazione
MQTT	Message Queuing Telemetry Transport — protocollo IoT leggero per sensori e telemetria
ACDAT	Ambiente di Condivisione Dati — il CDE ufficiale italiano per progetti BIM (ISO 19650)
CDE	Common Data Environment — piattaforma documentale condivisa per il progetto BIM
CDC	Change Data Capture — tecnica per catturare modifiche al database dal WAL
WAL	Write-Ahead Log — log delle modifiche PostgreSQL, base del CDC Debezium
CQRS	Command Query Responsibility Segregation — separazione modello scrittura / lettura
TPS	Totale Progressivo SAL — valore economico cumulativo certificato nel SAL corrente
TCS	Totale Contrattuale SAL — importo contrattuale di riferimento
RA_k	Rata di Acconto k-esima — importo da liquidare per il SAL k, calcolato con formula contrattuale
NC	Non Conformità — difformità rilevata rispetto ai requisiti tecnici o contrattuali
DURC	Documento Unico di Regolarità Contributiva — attestato di correttezza contributiva obbligatorio
RLS	Row Level Security — meccanismo PostgreSQL per l'isolamento dati a livello di riga
OIDC	OpenID Connect — protocollo di autenticazione basato su OAuth 2.0, usato da Keycloak
JWT	JSON Web Token — token firmato che trasporta identità e claims dell'utente
BFF	Backend For Frontend — layer server dedicato a un singolo frontend client

Nice to Have — MCP Server: Assistente AI Conversazionale

Nota: questa sezione descrive un'evoluzione futura non inclusa nel perimetro della versione corrente (MS-12). È una proposta tecnica completa da valutare in una fase successiva.

Il Problema che Risolve

Oggi, per rispondere alla domanda "Posso rilasciare il SAL 3 di Levante?", il Direttore Lavori deve navigare sequenzialmente tra 4-5 schermate: verificare l'avanzamento per elemento (Fase 8), controllare le non conformità aperte (sezione Qualità), verificare la copertura documentale (DDT, certificati), incrociare le condizioni contrattuali (Fase 5), guardare i numeri del SAL Engine (Fase 9). Il processo richiede 10-15 minuti e presuppone familiarità con l'interfaccia.

Un assistente AI conversazionale integrato permette di ottenere la stessa risposta in 2-4 secondi: l'utente scrive la domanda in linguaggio naturale, il sistema incrocia autonomamente tutti i dati e restituisce una risposta strutturata con evidenze specifiche. Il valore non è sostituire le dashboard ma aggiungere un canale di accesso rapido per le domande frequenti.

Architettura del Sistema MCP

Componenti

Il sistema si compone di tre componenti, ognuno con responsabilità distinte:

Componente	Tecnologia	Responsabilità
Chat Widget	React + EventSource (SSE) + Zustand	Pannello drawer lato destro, streaming token-by-token, conversation history, suggerimenti contestuali, formattazione markdown
Chat BFF	Python FastAPI + httpx-sse + anthropic/openai SDK	Orchestratore: riceve domanda, chiama LLM con catalogo tool, esegue tool call verso MCP Server, restituisce risposta in streaming SSE
MCP Server (MS-12)	Python FastAPI + mcp-python SDK	Espone 18 tool MCP stateless. Ogni tool esegue query business su Hasura/PostgreSQL e restituisce dati strutturati

Flusso di una Domanda

- Il widget React invia la domanda + JWT + conversation history al Chat BFF (`POST /chat`).
- Il BFF valida il JWT, filtra il catalogo tool in base al ruolo utente e chiama l'LLM.
- L'LLM decide quali tool invocare (`tool_use`) con quali parametri.
- Il BFF chiama l'MCP Server che esegue le query sul DB con `tenant_id` propagato.
- Il BFF passa i risultati all'LLM che compone la risposta in linguaggio naturale.
- La risposta arriva al widget in streaming SSE (token-by-token).

Latenza tipica: 1.5–3 secondi per domande semplici (1 tool call); 2–4 secondi per domande complesse (`get_sal_release_readiness` con verifica cross-schema).

Perché MCP e non function calling diretto

Il Model Context Protocol standardizza l'interfaccia tra LLM e strumenti. Vantaggi rispetto al function calling proprietario: portabilità tra provider LLM (lo stesso catalogo funziona con Claude, Gemini e GPT senza modifiche), possibilità futura di connessione da Claude Desktop o ChatGPT plugin senza modificare il server, e un pattern consolidato per discovery, versionamento e autenticazione dei tool.

Perché Chat BFF separato dall'MCP Server

(1) **Separazione responsabilità:** il BFF gestisce sessioni, streaming, history e orchestrazione LLM; il MCP Server espone solo tool stateless. (2) **Sicurezza:** il BFF è l'unico punto che parla con l'LLM esterno, centralizzando la sanitizzazione dei dati in uscita. (3) **Riusabilità:** il MCP Server può essere esposto a client esterni senza modifiche.

Catalogo Tool MCP (18 tool su 6 domini)

Dominio Avanzamento / SAL

Tool	Parametri chiave	Ritorno	Query sottostante
<code>get_sal_status</code>	project_id, sal_number?	Stato SAL: TPS, TCS, %, stato corrente, RA_k	<code>progress.SAL JOIN SAL_LINE_ITEM JOIN process.PAYMENT_CONDITION</code>
<code>get_sal_release_readiness</code>	project_id, sal_number	Check completo: quantità, saldi, doc, NC, condizioni contrattuali	Cross-schema (logica SAL Engine read-only)
<code>get_element_progress</code>	project_id, element_id?	Avanzamento per elemento: qty target vs done, stato, data	<code>progress.ELEMENT_PROGRESS JOIN bim.BIM_ELEMENT JOIN bim.BIM_QUANTITY</code>
<code>get_blocked_elements</code>	project_id	Lista elementi bloccati con motivo (NC, doc mancante)	<code>progress.ELEMENT_PROGRESS JOIN quality.NON_CONFORMITY</code>
<code>get_progress_timeline</code>	project_id, date_from?, date_to?	Serie temporale avanzamento % per settimana/mese	<code>progress.WORK_PROGRESS</code> aggregato per data

Dominio BIM / WBS

Tool	Parametri chiave	Ritorno	Query sottostante
<code>get_wbs_summary</code>	project_id	Riepilogo WBS: totale elementi, per fase, per categoria, quantità aggregate	<code>bim.BIM_ELEMENT + BIM_QUANTITY + CONSTRUCTION_PHASE</code> aggregati
<code>get_element_detail</code>	project_id, element_id	Dettaglio elemento: categoria, quantità, attività, avanzamento, produzione, qualità	Cross-schema per element_id
<code>search_elements</code>	project_id, query	Ricerca fuzzy su categoria/famiglia/tipo/livello	<code>bim.BIM_ELEMENT</code> con <code>ILIKE</code> / full-text search

Dominio Produzione

Tool	Parametri chiave	Ritorno	Query sottostante
<code>get_production_summary</code>	project_id, date_from?, mix_code?	Totale m³ per mix design, per periodo, per impianto	<code>production.PRODUCTION_RECORD + MIX_RECIPE</code> aggregati
<code>get_production_vs_design</code>	project_id, element_id?	Quantità prodotte vs quantità BIM, scostamento %	<code>production.PRODUCTION_RECORD</code> vs <code>bim.BIM_QUANTITY</code>

Tool	Parametri chiave	Ritorno	Query sottostante
<code>get_batch_trace</code>	<code>batch_id</code>	Tracciabilità completa: ricetta → autobetoniera → elemento WBS → SAL	Catena <code>PRODUCTION_RECORD</code> → <code>ELEMENT_PROGRESS</code> → <code>BIM_ELEMENT</code> → <code>SAL</code>

Dominio Qualità

Tool	Parametri chiave	Ritorno	Query sottostante
<code>get_non_conformities</code>	<code>project_id</code> , <code>status?</code> , <code>severity?</code>	Lista NC con descrizione, severità, stato, elemento WBS	<code>quality.NON_CONFORMITY JOIN progress.WORK_PROGRESS</code>
<code>get_quality_certificates</code>	<code>project_id</code> , <code>sal_number?</code>	Certificati per SAL: presenti vs attesi, tipo test, esito	<code>quality.QUALITY_TEST_CERT JOIN progress.WORK_PROGRESS</code>
<code>get_missing_documents</code>	<code>project_id</code> , <code>sal_number?</code>	DDT e certificati mancanti per SAL corrente	<code>document.DOCUMENT_REF</code> attesi vs presenti

Dominio Contratto / Processo

Tool	Parametri chiave	Ritorno	Query sottostante
<code>get_contract_conditions</code>	<code>project_id</code>	Soglia SAL, formula IS, tempi certificato/pagamento, DURC, penali	<code>process.CONTRACT + CLAUSE + PAYMENT_CONDITION</code>
<code>get_milestones</code>	<code>project_id</code> , <code>status?</code>	Milestone con data target, stato, % completamento	<code>process.MILESTONE + GANTT_ACTIVITY</code>
<code>get_payment_forecast</code>	<code>project_id</code>	Previsione pagamenti: SAL prossimi, importi stimati, date attese	<code>progress.SAL + process.PAYMENT_CONDITION + proiezione</code>

Dominio Portfolio / Cross-progetto

Tool	Parametri chiave	Ritorno	Query sottostante
<code>get_portfolio_kpi</code>	(nessuno)	Progetti attivi, valore totale, SAL rilasciati, avanzamento medio	<code>read.portfolio_kpi</code> (vista materializzata)
<code>get_projects_list</code>	<code>status?</code>	Lista progetti con stato, valore contratto, avanzamento, ultimo SAL	<code>read.project_summary</code>

Il Tool Composito: `get_sal_release_readiness`

Il tool più importante del sistema. Esegue in una singola chiamata la stessa logica dei 6 check del SAL Engine in modalità read-only. È progettato per evitare catene di 5-6 tool call sequenziali che richiederebbero 3 round-trip LLM aggiuntivi (6–10 secondi in più). Risposta strutturata:

```
{
  "sal_number": 3,
  "project": "Progetto Levante",
  "overall_status": "NOT_READY",
  "checks": {
    "quantity_cross_check": { "status": "WARNING", "match_rate": 0.987,
      "discrepancies": [{"element": "E_FND_001", "bim_qty": 3.85, "production_qty": 3.50}] },
    "economic_balance": { "status": "OK", "tps": 3780301, "tcs": 4874115, "ra_k": 750000 },
    "document_completeness": { "status": "OK", "coverage": 0.98,
      "missing": ["DDT autobetoniera BT-445"] },
    "non_conformities": { "status": "BLOCKING", "open_major": 1,
      "details": [{"id": "NC-0023", "severity": "Major", "element": "E_FND_003"}] },
    "contract_conditions": { "status": "OK", "threshold_met": true, "durc_valid": true }
  },
  "recommendation": "SAL 3 non rilasciabile: 1 NC Major aperta (NC-0023). Risolvere prima del rilascio."
}
```

Modello di Sicurezza

Propagazione JWT e Tenant

Il JWT Keycloak dell'utente viene propagato in ogni chiamata lungo la catena Widget → BFF → MCP Server. L'MCP Server inietta `tenant_id` come filtro obbligatorio in ogni query SQL/GraphQL. PostgreSQL RLS funge da safety net: anche se un bug nel codice MCP dimenticasse il filtro, RLS blocca l'accesso cross-tenant.

Filtraggio Tool per Ruolo

Ruolo	Tool accessibili	Restrizioni
<code>admin</code> , <code>project_manager</code> , <code>rup</code>	Tutti i 18 tool	Nessuna
<code>dl</code> (Direttore Lavori)	Tutti tranne portfolio cross-progetto su progetti non assegnati	<code>get_portfolio_kpi</code> solo propri progetti
<code>site_supervisor</code>	<code>get_element_progress</code> , <code>get_production_summary</code> , <code>get_production_vs_design</code> , <code>get_batch_trace</code> , <code>get_sal_status</code> , <code>search_elements</code>	No tool economici/contrattuali
<code>quality_manager</code>	<code>get_non_conformities</code> , <code>get_quality_certificates</code> , <code>get_missing_documents</code> , <code>get_element_detail</code> , <code>get_sal_status</code>	Solo visualizzazione NC e certificati

Protezione Dati verso LLM Esterno

L'LLM esterno riceve: domanda utente, catalogo tool (solo nomi e parametri), risultati delle tool call sanitizzati. Non riceve mai: JWT, credenziali, payload completi. Il BFF tronca i risultati a max 4.000 token per tool call prima di inviarli all'LLM e logga ogni interazione nella tabella `TOOL_CALL_LOG` per audit. Rate limiting: `site_supervisor` 30 domande/ora, `dl`/`rup` 100/ora. Max 10 tool call per singola domanda (evita loop LLM costosi).

Widget React: Suggerimenti Contestuali per Schermata

Schermata (Fase)	Suggerimenti auto-proposti
Fase 2 — Portfolio	"Qual è il progetto più avanzato?" · "Quali progetti hanno SAL bloccati?" · "Avanzamento medio portfolio?"
Fase 3 — Scheda Progetto	"Riepilogo stato SAL" · "Previsione prossimo pagamento?" · "Ritardi rispetto al Gantt?"

Schermata (Fase)	Suggerimenti auto-proposti
Fase 7 — Digital Twin	"Quanti elementi sono certificati?" · "Quali zone sono bloccate?" · "Avanzamento per fase?"
Fase 8 — Progress	"Quanti m³ gettati oggi?" · "Stato elemento E_FND_001?" · "Scostamento produzione vs progetto?"
Fase 9 — SAL Engine	"Posso rilasciare questo SAL?" · "NC bloccanti?" · "Documenti mancanti?" · "Dettaglio parificazione"

Il contesto della schermata corrente viene iniettato automaticamente nel system prompt del LLM. In Fase 9, una domanda come "Ci sono problemi?" viene automaticamente interpretata come "Ci sono NC aperte o documenti mancanti per il SAL corrente del progetto corrente?" senza che l'utente debba specificare project_id o sal_number.